COMPUTER VISION

Analisis Mendalam Topik Lightweight YOLO26n-Depth



Nama Dosen:

Prof. Dr. Eng. Fitri Utaminingrum, S.T., M.T.

Matakuliah & Kelas

Computer Vision - Kelas C

FAKULTAS ILMU KOMPUTER

PROGRAM STUDI TEKNIK KOMPUTER

UNIVERSITAS BRAWIJAYA

TAHUN 2026

KELOMPOK 1.1
Lightweight YOLO26-Depth

NAMA: Lutfi Maulana (NIM. 245150301111021)
TUGAS: Konsep, teori, dan dataset.

NAMA : Muhammad Dzakwan Ikram (NIM. 245150301111019)
TUGAS: Bedah arsitektur teknis untuk implementasi di kamera HP.

1.1. YOLO

YOLO (You Only Look Once) adalah algoritma kecerdasan buatan yang dirancang khusus untuk melakukan *object detection* secara otomatis pada gambar maupun video. Berbeda dengan klasifikasi biasa yang hanya menebak isi gambar, YOLO bisa mengenali jenis objek sekaligus menandai lokasinya menggunakan *bounding box* (Redmon dkk., 2016)..

Prinsip kerjanya sesuai dengan namanya, yaitu memproses seluruh piksel gambar secara bersamaan hanya dalam satu kali penelusuran. Dengan cara ini YOLO jadi sangat cepat dan mampu memahami konteks visual secara utuh tanpa perlu memotong gambar menjadi bagian-bagian kecil terlebih dahulu.

1.2. YOLO V26

YOLO (You Only Look Once) V26 adalah generasi terbaru dari arsitektur computer vision yang dikembangkan oleh Ultralytics (Ultralytics, 2026). Pembaruan paling revolusioner pada versi ini adalah integrasi fitur monocular depth estimation (estimasi kedalaman dari satu kamera) sebagai fungsi bawaan (Ultralytics, 2026). Sebelumnya, penambahan fitur deteksi jarak pada pipeline YOLO mengharuskan penggunaan model tambahan dari proyek terpisah (seperti MiDaS) atau membutuhkan perangkat keras khusus seperti kamera ganda (stereo) dan sensor LiDAR (Ultralytics, 2026). Kini, YOLO V26 mampu memprediksi jarak metrik absolut dalam satuan meter hanya dengan mengandalkan masukan dari satu citra kamera RGB biasa (Ultralytics, 2026).

Kelebihan lain yang signifikan pada YOLO V26 adalah kemampuannya mendeteksi jarak tanpa batas (unbounded depth). Jika model lain membatasi rentang prediksi maksimal sekitar 10 meter, YOLO V26 menggunakan skala logaritma yang memungkinkannya beroperasi secara akurat untuk skenario jarak dekat di dalam ruangan, maupun skenario jarak jauh di luar ruangan dengan objek berjarak puluhan meter (Ultralytics, 2026). Selain itu, arsitektur ini direkonstruksi secara mendalam agar ukurannya jauh lebih ringan, yang mana sangat krusial untuk

memangkas beban komputasi saat diimplementasikan pada perangkat edge atau terminal komputasi dengan sumber daya yang sangat terbatas (Ning, dkk., 2026). Arsitektur lightweight ini dirancang khusus untuk menjaga keseimbangan optimal antara akurasi deteksi yang tinggi dan kecepatan komputasi (real-time processing) (Zare, dkk., 2026).

Untuk penerapannya secara langsung, model ini sangat ideal digunakan pada perangkat mobile (seperti kamera smartphone). YOLO V26 menyediakan jalur ekspor yang terpadu ke berbagai format optimasi perangkat keras, seperti LiteRT atau NCNN untuk ekosistem Android, serta format CoreML untuk iPhone (iOS) (Ultralytics, 2026). Kemudahan lintas platform ini dikombinasikan dengan pemangkasan ukuran model memungkinkan proses inferensi peta kedalaman dilakukan secara langsung (real-time) di perangkat lokal tanpa membebani daya baterai dan tanpa bergantung pada pemrosesan server eksternal (Yuan, dkk., 2024).

1.2.1 ARSITEKTUR YOLO V26

Arsitektur dari varian lightweight YOLO V26 (seperti versi yolo26n-depth atau nano) dirancang khusus agar bisa bekerja maksimal dengan sumber daya perangkat yang terbatas (Ultralytics, 2026). Secara struktur, arsitektur ini menyederhanakan proses pemahaman gambar melalui tiga komponen utama, yaitu Backbone, Neck, dan Depth Head (Ultralytics, 2026).

- Backbone Feature Extraction (Ekstraksi Fitur Tulang Punggung): Bagian ini bertugas membaca dan mengekstrak ciri-ciri penting dari data masukan yang telah melewati tahap pra-pemrosesan (Preprocessing: Resize and Normalize) untuk kemudian dipecah menjadi peta fitur dari berbagai resolusi (Multi-Scale Feature Maps) (Ultralytics, 2026). Pada versi ini, ekstraksi fitur menggunakan metode perhitungan yang disederhanakan agar prosesnya berjalan sangat cepat dan tidak memakan banyak memori (Ultralytics, 2026).
- Lightweight Feature Fusion Neck (Leher Fusi Fitur

Ringan): Komponen ini berfungsi untuk menggabungkan informasi dari Multi-Scale Feature Maps secara efisien tanpa membebani komputasi (Ultralytics, 2026). Proses penggabungan skala ini sangat penting agar model dapat memahami konteks ruang secara utuh, sehingga ia bisa mengukur benda yang jaraknya sangat dekat di depan lensa maupun benda yang ada di kejauhan secara bersamaan (Ultralytics, 2026).

- Direct Regression Head (Kepala Regresi Langsung sebagai Depth Head): Berbeda dengan model YOLO biasa yang membutuhkan proses eliminasi kotak penanda tambahan, bagian ini dirancang untuk menghasilkan prediksi secara langsung tanpa hambatan (NMS-Free Predictions) (Ultralytics, 2026). Dalam fungsinya untuk mengukur jarak, Direct Regression Head bertugas menghasilkan peta kedalaman (depth map) yang menghitung perkiraan jarak pasti dalam satuan meter untuk setiap titik (piksel) pada layar secara langsung (Ultralytics, 2026).



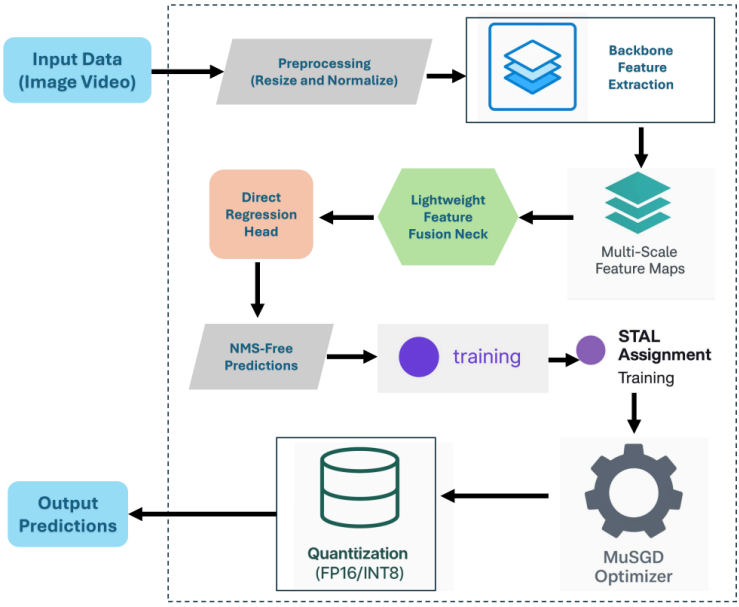
Sumber:
https://developers.google.com/ar/develop/java/depth/quickstart?hl=id

Gambar 1.1 Visualisasi Pasangan Citra RGB dan Monocular Depth Map pada Perangkat Mobile

Optimasi untuk Perangkat Mobile

Arsitektur YOLO V26 versi lightweight memangkas jumlah jaringannya di bawah 10 juta parameter menggunakan teknik kuantisasi dan network pruning tanpa mengorbankan akurasi (Ultralytics, 2026). Ukuran model yang ramping ini mengurangi beban prosesor serta kompatibel dengan Neural Processing Unit (NPU) modern untuk mengalihkan komputasi dari CPU (Ultralytics, 2026).

Dampaknya, ponsel membutuhkan lebih sedikit tenaga dan RAM untuk memproses gambar dengan transfer data yang sangat cepat dalam hitungan milidetik (Ultralytics, 2026). Beban komputasi yang ringan membuat aplikasi berjalan mulus secara real-time tanpa membuat perangkat cepat panas atau overheating, sehingga terhindar dari thermal throttling (Ultralytics, 2026). Pada akhirnya, daya baterai tetap awet meskipun pemrosesan AI dan kamera digunakan terus-menerus, membuka peluang integrasi ke dalam aplikasi seperti navigasi visual, pemindai ruang 3D, atau augmented reality (AR) (Ultralytics, 2026).



Sumber: https://learnopencv.com/yolov26-real-time-deployment/

Gambar 1.2 Gambar Core Arsitektur YOLO V26

1.3. DYNAMIC ATTENTION

Dynamic Attention adalah fitur pembantu yang membuat AI bisa memilih bagian gambar mana yang paling penting untuk diperhatikan. Pada versi lightweight, ukuran program YOLO26-Depth disederhanakan agar tidak berat saat dijalankan di Handphone. Namun, penyederhanaan ini bisa membuat AI kurang teliti dalam melihat detail. Dynamic Attention mengatasi masalah tersebut dengan cara mengarahkan fokus AI hanya ke bagian-bagian penting, seperti batas pinggir benda atau bentuk objek, daripada membuang tenaga memeriksa seluruh isi gambar. Hasilnya, HP tetap bisa mendeteksi objek dan mengukur jarak secara akurat tanpa membuat kinerja HP terasa lambat atau boros baterai.

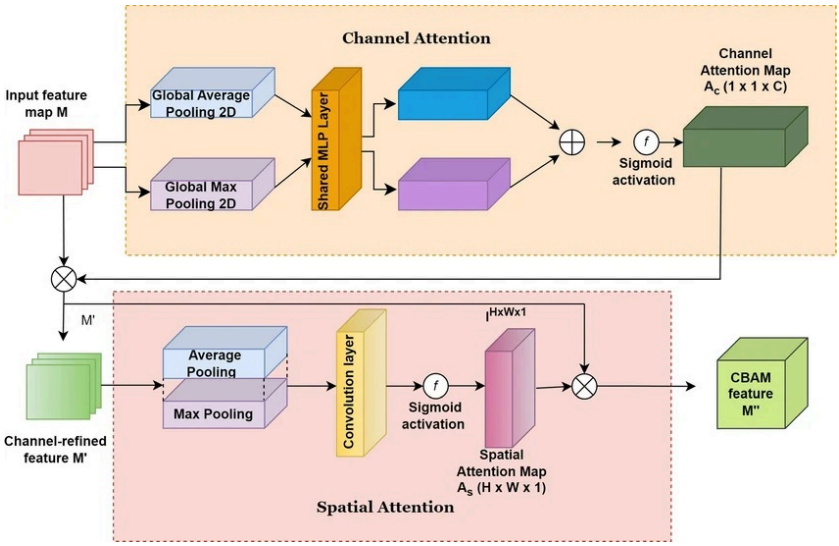
1.3.1 ARSITEKTUR DYNAMIC ATTENTION

Meskipun ukuran model telah dipangkas drastis melalui teknik kuantisasi dan pemangkasan jaringan untuk keperluan kamera HP, akurasi peta kedalaman dapat dipertahankan melalui integrasi mekanisme Dynamic Attention, seperti penerapan Convolutional Block Attention Module (CBAM) (Ultralytics, 2026). Proses pemangkasan jaringan yang dalam (deep lightweighting) rentan menyebabkan hilangnya fitur-fitur detail (fine-grained features) pada gambar (Ultralytics, 2026). Oleh karena itu, lapisan atensi ini berfungsi sebagai kompensasi dengan memandu jaringan untuk fokus pada area tepi atau kontur yang penting dan secara efektif menekan noise dari latar belakang (Ultralytics, 2026).

Mekanisme ini bekerja melalui dua modul dimensi utama secara berurutan:

- Channel Attention (Atensi Kanal): Berfokus pada penyeleksian fitur apa yang paling penting dengan mempelajari bobot setiap kanal pada peta fitur multi-skala secara adaptif (Ultralytics, 2026). Modul ini akan meningkatkan respons pada kanal yang memuat informasi bentuk secara valid dan menyaring informasi yang tidak relevan (Ultralytics, 2026).

- Spatial Attention (Atensi Spasial): Berfokus pada menganalisis di mana letak informasi penting tersebut berada di dalam gambar (Ultralytics, 2026). Modul ini menangkap ketergantungan ruang untuk mengkalibrasi distribusi target secara presisi pada gambar dua dimensi sebelum diteruskan ke bagian kepala regresi (Direct Regression Head) (Ultralytics, 2026).



Sumber: https://www.researchgate.net/figure/Architecture-of-Convolutional-Block-Attention-Module-CBAM-10_fig4_381431411

Gambar 1.3 Diagram Arsitektur Convolutional Block Attention Module (CBAM)

Melalui mekanisme atensi ganda ini, model dapat mempertahankan akurasi lokalisasi yang sangat tinggi dengan tambahan beban komputasi yang tergolong sangat minim (Ultralytics, 2026). Hal ini menjadikannya sangat andal untuk penggunaan real-time tanpa membuat prosesor smartphone bekerja terlalu keras atau memicu overheating (Ultralytics, 2026).

1.4 DATASET

Dalam pengembangan model Lightweight YOLO26-Depth, dataset berfungsi sebagai bahan latihan utama agar AI dapat mempelejari cara mengenali objek sekaligus mengestimasi jarak secara akurat. Karena model ini memiliki dua tugas utama (deteksi objek dan pengukuran kedalaman), proses pelatihannya membutuhkan kumpulan citra RGB yang dilengkapi dengan data jarak (ground truth depth).

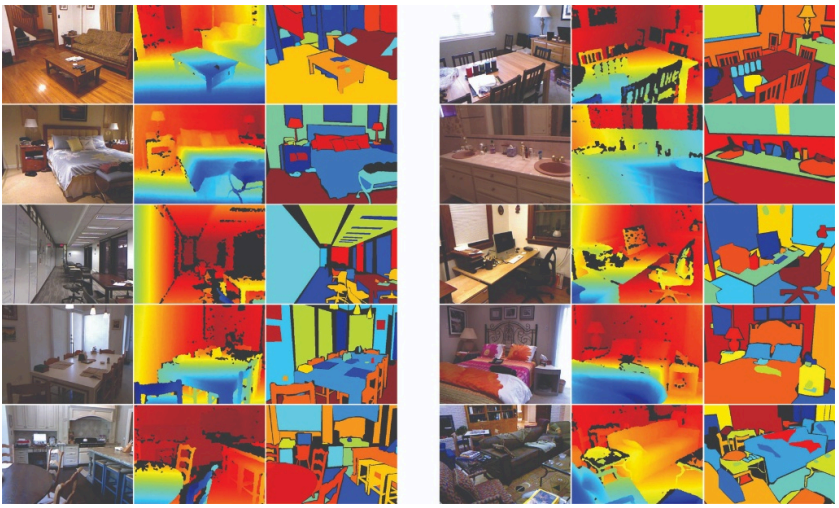
Beberapa dataset standar internasional yang umum digunakan untuk melatih model ini meliputi:

- NYU Depth V2: Dataset khusus lingkungan dalam ruangan (indoor) yang berisi ribuan pasangan foto RGB dan peta kedalaman. Dataset ini digunakan agar AI belajar mengenali jarak objek skala kecil hingga menengah dalam ruangan, seperti meja, kursi, atau dinding.

- KITTI Vision Benchmark: Dataset khusus lingkungan luar ruangan (outdoor) dan jalan raya yang direkam menggunakan kendaraan berkamera serta sensor LiDAR. Dataset ini melatih AI untuk memahami estimasi jarak skala luas (unbounded depth), seperti mengukur jarak kendaraan lain, pejalan kaki, atau bangunan di kejauhan.

- MS COCO (Common Objects in Context): Dataset skala besar yang berisi ratusan ribu gambar dengan berbagai variasi objek sehari-hari. Dataset ini dipakai pada tahap awal (pre-training) untuk melatih bagian Backbone agar AI mahir membuat bounding box dan mengenali berbagai jenis objek sebelum memprediksi jaraknya.

Melalui kombinasi dataset tersebut, model YOLO26-Depth yang sudah dirancang ringan (lightweight) tetap memiliki pemahaman konteks ruang yang kuat, sehingga siap digunakan pada berbagai skenario nyata langsung melalui kamera smartphone.



Sumber: https://cs.nyu.edu/~fergus/datasets/nyu_depth_v2.html

Gambar 1.4 Sampel Pasangan Citra RGB, Depth Map, dan Semantic Label pada Dataset NYU Depth V2

KELOMPOK 1.1
Lightweight YOLO26-Depth

NAMA: Lutfi Maulana (NIM. 245150301111021)
TUGAS: Merancang ringkasan alur kerja pipeline baseline (Sub-bab 2.1) serta mengintegrasikan kode pemanggilan model PyTorch dan arsitektur YOLO26nDepth (Sub-bab 2.2).

NAMA : Muhammad Dzakwan Ikram (NIM. 245150301111019)
TUGAS: Mengimplementasikan kode penanganan stream kamera dan visualisasi (Sub-bab 2.2) serta menyusun pembedahan detail logika dan fungsi program (Sub-bab 2.3).

2.1 Deskripsi Alur Pipeline Baseline

Program baseline ini dirancang untuk memperkirakan jarak atau kedalaman objek secara langsung (real-time) hanya menggunakan kamera biasa (webcam).Program ini menyediakan dua pilihan mode untuk menjalankan model:

- **Mode Lokal PyTorch (local):** Menjalankan model YOLO26nDepth buatan sendiri (rincian arsitekturnya dibahas di Bab 3) dengan memuat berkas bobot terlatih (.pt) langsung ke GPU atau CPU.
- **Mode Ultralytics (ultralytics):** Menjalankan model secara langsung menggunakan pustaka resmi bawaan Ultralytics.

Secara garis besar, alur kerja program ini berjalan terus-menerus melalui tahapan berikut:

Tangkap gambar kamera -> Siapkan format gambar (preprocessing) -> Prediksi jarak dengan model -> Rapikan hasil kedalaman (post-processing) -> Tampilkan video asli dan peta kedalaman secara berdampingan (side-by-side).

2.2 Kode Program Baseline

```
import argparse
import os

import cv2
import numpy as np
import torch

from arsitektur_yolo_depth import YOLO26nDepth


def normalize_depth(depth_map):
    depth_map = np.asarray(depth_map, dtype=np.float32)
    low = float(np.percentile(depth_map, 2.0))
    high = float(np.percentile(depth_map, 98.0))
    if high - low < 0.01:
        return np.zeros_like(depth_map, dtype=np.uint8)
    normalized = np.clip((depth_map - low) / (high - low), 0.0,
1.0)
```

```
    normalized = 1.0 - normalized
    return (normalized * 255.0).astype(np.uint8)


def colorize_depth(depth_map):
    return cv2.applyColorMap(normalize_depth(depth_map),
cv2.COLORMAP_INFERNO)


def load_local_model(weights_path):
    device = torch.device('cuda' if torch.cuda.is_available() else
'cpu')
    model = YOLO26nDepth(width_mult=0.25,
depth_mult=0.50).to(device)
    state_dict = torch.load(weights_path, map_location=device,
weights_only=False)
    model.load_state_dict(state_dict)
    model.eval()
    print(f'Backend: local PyTorch | Device: {device}')
    return model, device


def run_local_camera(weights_path, camera_index):
    model, device = load_local_model(weights_path)
    camera = open_camera(camera_index)

    try:
        while True:
            ok, frame = camera.read()
            if not ok:
                print('Gagal membaca frame dari kamera.')
                break

            rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            image = torch.from_numpy(rgb.transpose(2, 0,
1)).float().div(255.0)
            image = image.unsqueeze(0).to(device)
            with torch.no_grad():
                depth = model(image)['depth'][0, 0].cpu().numpy()

            depth_color = cv2.resize(colorize_depth(depth),
```

```
(frame.shape[1], frame.shape[0]))
        show_camera_result(frame, depth_color, 'YOLO26n-depth
Local')
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break
    finally:
        close_camera(camera)


def run_ultralytics_camera(weights_path, camera_index):
    from ultralytics import YOLO

    model = YOLO(weights_path)
    camera = open_camera(camera_index)
    print(f'Backend: Ultralytics | Model: {weights_path}')

    try:
        while True:
            ok, frame = camera.read()
            if not ok:
                print('Gagal membaca frame dari kamera.')
                break

            result = model(frame, verbose=False)[0]
            depth_result = getattr(result, 'depth', None)
            if depth_result is None:
                raise RuntimeError(
                    'Model tidak menghasilkan result.depth. Pastikan
checkpoint adalah model depth Ultralytics.'
                )

            depth = depth_result.data.detach().cpu().numpy()
            if depth.ndim == 3:
                depth = depth[0]
            depth_color = cv2.resize(colorize_depth(depth),
(frame.shape[1], frame.shape[0]))
            show_camera_result(frame, depth_color, 'YOLO26n-depth
Ultralytics')
            if cv2.waitKey(1) & 0xFF == ord('q'):
                break
    finally:
```

```
        close_camera(camera)


def open_camera(camera_index):
    camera = cv2.VideoCapture(camera_index, cv2.CAP_DSHOW)
    if not camera.isOpened():
        camera = cv2.VideoCapture(camera_index)
    if not camera.isOpened():
        raise RuntimeError(f'Kamera index {camera_index} tidak
dapat dibuka.')
    print("'Tekan 'q' untuk keluar.")
    return camera


def close_camera(camera):
    camera.release()
    cv2.destroyAllWindows()


def show_camera_result(frame, depth_color, title):
    combined = np.hstack((frame, depth_color))
    cv2.imshow(title, combined)


def detect_backend(weights_path, requested_backend):
    if requested_backend != 'auto':
        return requested_backend
    filename = os.path.basename(weights_path).lower()
    if filename in {'best.pt', 'yolo26n-depth.pt'}:
        return 'ultralytics'
    return 'local'


if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Unified
YOLO26n-depth camera runner')
    parser.add_argument('--weights',
default='data/yolo26n_depth_nyu_full.pt')
    parser.add_argument('--backend', choices=['auto', 'local',
'ultralytics'], default='auto')
    parser.add_argument('--camera', type=int, default=0)
```

```
    args = parser.parse_args()

    if not os.path.exists(args.weights):
        raise SystemExit(f'Checkpoint tidak ditemukan:
{args.weights}')

    backend = detect_backend(args.weights, args.backend)
    if backend == 'local':
        run_local_camera(args.weights, args.camera)
    else:
        run_ultralytics_camera(args.weights, args.camera)
```

Sumber: Penulis (Disesuaikan dengan arsitekur YOLOv26n)

2.3 Penjelasan Logika dan Fungsi Program

Bagian ini menguraikan fungsi kerja dari setiap komponen utama pada kode program:

- **Normalisasi Kedalaman (normalize_depth):**
  Fungsi ini mengubah angka estimasi jarak dari model menjadi format gambar biasa dengan rentang nilai 0 hingga 255. Nilai 2% terendah dan 2% tertinggi dipotong menggunakan persentil agar gambar tidak rusak oleh pantulan cahaya silau atau bayangan gelap. Nilainya kemudian dibalik (1.0 - normalized) agar benda yang berada dekat dengan kamera tampak lebih terang, sedangkan latar belakang yang jauh tampak lebih gelap.
- **Pewarnaan Peta Kedalaman (colorize_depth):**
  Fungsi ini memberi warna pada gambar hitam-putih hasil normalisasi menggunakan skema COLORMAP_INFERNO dari OpenCV. Hasilnya, pembagian jarak terlihat lebih jelas: objek yang sangat dekat berwarna kuning atau oranye cerah, jarak sedang berwarna merah keunguan, dan area yang paling jauh berwarna ungu tua hingga hitam.
- **Inisialisasi Model Lokal (load_local_model):**
  Fungsi ini menyiapkan model YOLO26nDepth varian nano (width_mult=0.25, depth_mult=0.50) dan menempatkannya ke GPU atau CPU. File bobot yang telah dilatih kemudian dimuat ke dalam model, lalu sistem

diatur ke mode evaluasi (model.eval()) agar pengujian berjalan konsisten.

- **Alur Pemrosesan Kamera (run_local_camera):**
  Fungsi ini membaca rekaman kamera secara langsung frame demi frame:
  - Mengubah format warna gambar dari BGR ke RGB serta menyusun susunan datanya agar sesuai dengan format tensor PyTorch.
  - Mematikan perhitungan gradien menggunakan torch.no_grad() saat memprediksi jarak agar komputasi lebih ringan dan tampilan video tetap lancar.
  - Mengambil hasil prediksi jarak pada bagian ['depth'], menyamakan ukurannya kembali dengan resolusi asli kamera, lalu menampilkan video asli dan peta kedalaman secara berdampingan di satu layar.
- **Pemilihan Backend Otomatis (detect_backend):**
  Fungsi ini memeriksa nama file bobot yang dimasukkan. Jika nama file terdeteksi sebagai model resmi Ultralytics, sistem otomatis memakai fungsi run_ultralytics_camera. Jika file merupakan bobot PyTorch lokal (data/yolo26n_depth_nyu_full.pt), sistem langsung menjalankan fungsi run_local_camera.

KELOMPOK 1.1
Lightweight YOLO26-Depth

NAMA: Lutfi Maulana (NIM. 245150301111021)
TUGAS: Menyusun ringkasan konfigurasi spesifikasi model baseline YOLO26n (compound scaling, backbone, neck, head) serta menganalisis diagram alur arsitektur dan titik modifikasinya (Sub-bab 3.1).

NAMA : Muhammad Dzakwan Ikram (NIM. 245150301111019)
TUGAS: Memverifikasi kesesuaian implementasi kode program PyTorch dengan rancangan arsitektur, mulai dari blok konvolusi dasar hingga integrasi modul DepthHead dan fungsi forward (Sub-bab 3.2).

3.1 Spesifikasi Arsitektur Baseline (YOLO26n)

Dengan menyesuaikan judul tugas yang diberikan, kami mendapatkan model YOLO26n sebagai arsitektur dasar (baseline). Kami menyimpulkan bahwa varian Nano (n) dipilih karena dirancang untuk keseimbangan optimal antara kecepatan komputasi dan akurasi, menjadikannya sangat efisien untuk proses deteksi secara real-time. Spesifikasi lengkap konfigurasi arsitektur dasar YOLO26n dirangkum pada Tabel 3.1.

Tabel 3.1 Ringkasan Konfigurasi Arsitektur YOLO26n (Baseline)

Komponen	Layer / Sumber	Modul Utama	Output / Fungsi
Compound Scaling	Varian YOLO26n	depth = 0.50, width = 0.25, max channels = 1024	260 layer, 2.572.280 parameter, 6.1 GFLOPs
Backbone	Layer 0–10	Conv, C3k2, SPPF, C2PSA	Ekstraksi fitur hierarkis dari skala P1/2 hingga P5/32
Fitur P3	Layer 3–4 (digunakan kembali di Neck)	Conv dan C3k2	Menyediakan fitur spasial detail untuk deteksi objek berukuran kecil (small objects)
Fitur P4	Layer 5–6 (digunakan kembali di Neck)	Conv dan C3k2	Menyediakan fitur tingkat menengah untuk objek berukuran sedang (medium objects)

Fitur P5	Layer 7–10	Conv, C3k2, SPPF, C2PSA	Menyediakan fitur semantik tingkat tinggi untuk objek berukuran besar (large objects)
Neck	Layer 11–22	Upsample, Concat, Conv, C3k2	Menggabungkan (aggregate) fitur multi-skala dari alur backbone dan head
Detection Head	Layer 23 (dari Layer 16, 19, 22)	Detect	Memprediksi bounding box dan label kelas pada skala P3/8, P4/16, dan P5/32
Task Setting	Layer Detect Akhir	Detect [jumlah kelas]	Jumlah kasus/output disesuaikan dengan kelas target pada konfigurasi data.yaml

Sumber: Ardhi, Hartono, dan Yoeseph, 2026.

3.1.1 Skala Komputasi (Compound Scaling):

Arsitektur YOLO26n dikonfigurasi menggunakan depth factor sebesar 0.50 dan width factor sebesar 0.25 dengan batas saluran maksimum 1024 channel. Konfigurasi ini menghasilkan model yang ringan dengan total 260 layer, 2.572.280 parameter, serta beban komputasi sebesar 6.1 GFLOPs.

Jika kita mengubah angka depth atau width ini (misalnya menjadi 1,00), varian modelnya akan berubah secara otomatis dari Nano (n) menjadi varian yang lebih besar seperti Small (s), Medium (m), atau Large (l).

3.1.2 Ekstraksi Fitur (Backbone):

Bagian backbone beroperasi pada Layer 0 hingga 10 menggunakan kombinasi konvolusi standar (Conv), blok C3k2, SPPF (Spatial Pyramid Pooling - Fast), dan C2PSA (Cross Stage Partial with Attention). Backbone mengekstrak representasi citra secara hierarkis yang terbagi menjadi tiga tingkatan resolusi:

- Fitur P3 (Layer 3–4): Mempertahankan informasi spasial mendalam yang berguna untuk mendeteksi objek berukuran kecil.
- Fitur P4 (Layer 5–6): Mengambil fitur tekstur tingkat menengah untuk objek berukuran sedang.
- Fitur P5 (Layer 7–10): Menangkap informasi semantik tingkat tinggi untuk membedakan objek berukuran besar.

Conv (Convolution): Bertugas melakukan downsampling (memperkecil resolusi spasial) sekaligus mengekstrak fitur dasar.

C3k2: Blok pemroses utama berbasis Cross Stage Partial (CSP) yang efisien mengekstrak fitur tanpa membengkakkan beban memori.

SPPF (Spatial Pyramid Pooling - Fast): Menggabungkan peta fitur dari berbagai skala pooling secara cepat untuk memperluas area pandang (receptive field) model.

C2PSA (Cross Stage Partial with Attention): Blok bawaan YOLO26n yang sudah dilengkapi self-attention internal untuk menangkap hubungan spasial antarfitur sebelum dilempar ke bagian Neck.

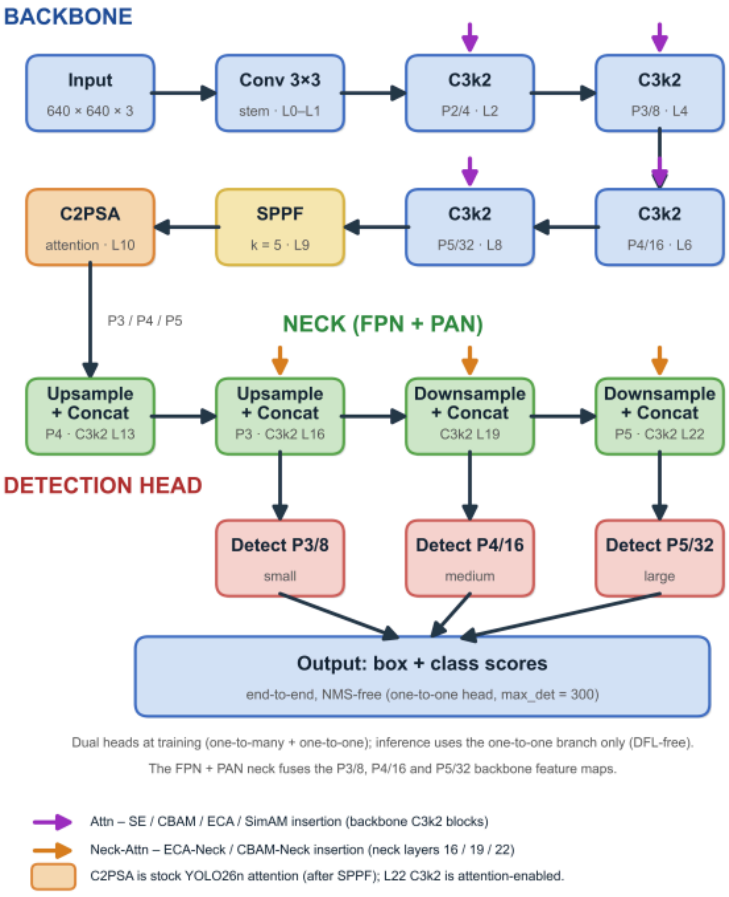
3.1.3 Penggabungan Fitur Multi-Skala (Neck):

Terletak pada Layer 11 hingga 22, komponen Neck memanfaatkan fungsi Upsample, Concat, Conv, dan C3k2 untuk menggabungkan peta fitur dari berbagai resolusi yang dihasilkan oleh backbone. Hal ini memastikan bahwa fitur spasial detail dan fitur semantik kaya dapat saling melengkapi sebelum diteruskan ke tahap prediksi.

3.1.4 Prediksi Akhir (Detection Head & Task Setting):

Detection Head berada pada Layer 23 yang menerima masukan dari layer 16, 19, dan 22 (sesuai dengan skala P3/8, P4/16, dan P5/32). Komponen ini menghasilkan output prediksi posisi bounding box dan skor klasifikasi secara langsung (NMS-free). Pada tahap Task Setting, jumlah output layer Detect disesuaikan secara dinamis mengikuti alokasi kelas yang didefinisikan dalam berkas data.yaml.

3.1.5 Alur Arsitektur dan Modifikasi Model



Sumber: Burukanli, Cibuk, dan Ari, 2026.

Gambar 3.1 Arsitektur Model YOLO26n

Diagram ini menggambarkan perjalanan citra dari input hingga ke hasil prediksi:

- Backbone (Ekstraksi Fitur):
    - Input: Citra masukan ukuran 640 x 640 x 3.
    - Conv 3x3 (L0–L1): Pemrosesan awal (*stem layer*).
    - C3k2 (L2, L4, L6, L8): Ekstraksi fitur bertahap dari skala P2/4 hingga P5/32.
    - SPPF (L9): *Spatial Pyramid Pooling - Fast* untuk memperluas area pandang fitur.
    - C2PSA (L10): Layer penutup *backbone*.
- Neck / FPN + PAN (Penggabungan Fitur):
    - Menggabungkan fitur dari *backbone* melalui proses *Upsample*, *Downsample*, dan Concat pada Layer 13, 16, 19, dan 22.
- Detection Head (Prediksi Akhir):
    - Detect P3/8 untuk deteksi objek kecil (*small*).
    - Detect P4/16 untuk deteksi objek sedang (*medium*).
    - Detect P5/32 untuk deteksi objek besar (*large*).
    - Menghasilkan *output bounding box* dan skor kelas secara *end-to-end / NMS-free*.

Karena ini kami mengambil dari salah satu jurnal yang meneliti tentang YOLOv26n tapi dengan modifikasi di arsitektur, maka kami langsung membaginya seperti ini:

Semua kotak berwarna (biru, hijau, merah, kuning, oranye) beserta garis panah hitam di dalamnya adalah arsitektur standar YOLO26n.

Bagian yang dimodifikasi adalah :

- Panah Ungu pada Backbone: Menandakan titik di mana peneliti memasukkan modul attention tambahan (pilihan: SE, CBAM, ECA, atau SimAM) ke dalam blok-blok C3k2 di Backbone (Layer 2, 4, 6, dan 8).

- Panah Jingga pada Neck: Menandakan titik penyisipan modul attention (pilihan: ECA-Neck atau CBAM-Neck) pada layer keluaran Neck (Layer 16, 19, dan 22).

Sebenarnya modifikasi ini tidak bertolak belakang dengan yang kami inginkan karena ini berfungsi sebagai filter fitur yang secara langsung mengubah karakteristik peta fitur (*feature map*) dan visualisasi *heatmap* menjadi lebih presisi dalam mengunci objek target sekaligus meredam gangguan latar belakang (*background noise*). Hasilnya, luaran deteksi akhir mengalami peningkatan akurasi (mAP) dan performa yang lebih andal untuk objek berukuran kecil atau terhalang, dengan kompensasi penambahan parameter komputasi serta latensi yang relatif minimal.

Tapi karena ini tidak termasuk ke dalam baseline arsitektur jadi kami abaikan.

3.2 Penjelasan Kesesuaian Program dengan Arsitektur

1. ConvBlock

ConvBlock merupakan blok dasar yang digunakan pada hampir seluruh bagian arsitektur YOLO26n-Depth. Blok ini terdiri atas convolution, batch normalization, dan fungsi aktivasi SiLU. Convolution berfungsi untuk mengekstraksi berbagai karakteristik visual dari citra, seperti garis, tekstur, bentuk, dan pola. Batch normalization digunakan untuk menjaga kestabilan distribusi nilai fitur selama proses training. Sementara itu, fungsi aktivasi SiLU memberikan kemampuan non-linear sehingga model dapat mempelajari hubungan yang lebih kompleks dari data masukan.

```
class ConvBlock(nn.Module):
  def __init__(self, in_ch, out_ch, k=3, s=1, p=1, g=1):
    super().__init__()
    self.conv = nn.Conv2d(
      in_ch,
      out_ch,
      kernel_size=k,
```

```
        stride=s,
        padding=p,
        groups=g,
        bias=False
    )
    self.bn = nn.BatchNorm2d(out_ch)
    self.act = nn.SiLU()
  def forward(self, x):
    return self.act(self.bn(self.conv(x)))
```

2. Bottleneck

Bottleneck merupakan blok yang terdiri atas dua lapisan convolution dengan jumlah channel yang disesuaikan dengan fitur masukan. Hasil pemrosesan kemudian digabungkan dengan input awal menggunakan koneksi residual. Koneksi residual memungkinkan informasi dari input diteruskan secara langsung ke bagian output sehingga membantu mempertahankan informasi fitur selama proses ekstraksi. Struktur ini juga membantu menjaga kestabilan proses training dan mengurangi permasalahan pada propagasi gradien. Bottleneck digunakan sebagai salah satu komponen penyusun pada modul C3k2.

```
class Bottleneck(nn.Module):
  def __init__(self, ch):
    super().__init__()
    self.cv1 = ConvBlock(ch, ch, k=3, s=1, p=1)
    self.cv2 = ConvBlock(ch, ch, k=3, s=1, p=1)

  def forward(self, x):
    return x + self.cv2(self.cv1(x))
```

3. C3k2

C3k2 merupakan modul ekstraksi fitur yang membagi feature map masukan menjadi dua jalur pemrosesan. Salah satu jalur mempertahankan sebagian informasi fitur secara langsung, sedangkan jalur lainnya diproses melalui beberapa blok Bottleneck untuk menghasilkan fitur yang lebih kompleks. Setelah kedua jalur selesai diproses, hasilnya digabungkan menggunakan operasi concatenation. Feature map hasil penggabungan kemudian diproses kembali menggunakan convolution untuk menghasilkan keluaran akhir. Struktur tersebut memungkinkan model memanfaatkan informasi fitur awal sekaligus fitur yang telah mengalami ekstraksi lebih lanjut.

```
class C3k2(nn.Module):
  def __init__(self, c1, c2, n=1):
    super().__init__()
    c_ = c2 // 2
    self.cv1 = ConvBlock(c1, c_, k=1, s=1, p=0)
    self.cv2 = ConvBlock(c1, c_, k=1, s=1, p=0)
    self.m = nn.ModuleList([Bottleneck(c_) for _ in range(n)])
    self.cv3 = ConvBlock(2 * c_, c2, k=1, s=1, p=0)

  def forward(self, x):
    y1 = self.cv1(x)
    y2 = self.cv2(x)

    for m in self.m:
      y2 = m(y2)

    return self.cv3(torch.cat((y1, y2), dim=1))
```

4. C2PSA

C2PSA merupakan modul attention yang digunakan pada bagian akhir backbone. Modul ini mempelajari tingkat kepentingan fitur dari feature map menggunakan mekanisme adaptive average pooling dan convolution. Informasi tersebut kemudian digunakan untuk menghasilkan bobot perhatian yang diterapkan pada fitur masukan. Fitur yang dianggap lebih relevan akan memperoleh perhatian yang lebih besar, sedangkan fitur yang kurang relevan dapat ditekan. Dengan mekanisme tersebut, model dapat lebih fokus pada informasi visual yang mendukung proses deteksi objek dan estimasi kedalaman.

```
class C2PSA(nn.Module):
  def __init__(self, c1, c2):
    super().__init__()
    self.cv1 = ConvBlock(c1, c2, k=1, s=1, p=0)
    self.attn = nn.Sequential(
      nn.AdaptiveAvgPool2d(1),
      nn.Conv2d(c2, c2 // 4, 1),
      nn.SiLU(),
      nn.Conv2d(c2 // 4, c2, 1),
      nn.Sigmoid(),
    )

  def forward(self, x):
    x = self.cv1(x)
    return x * self.attn(x)
```

5. SPPF

SPPF (Spatial Pyramid Pooling Fast) digunakan untuk memperluas receptive field model sehingga model dapat memperoleh informasi konteks dari area citra yang lebih luas. Modul ini menerapkan operasi max pooling secara bertahap dengan beberapa ukuran konteks yang diperoleh melalui proses pooling berurutan. Hasil dari proses tersebut kemudian digabungkan dengan feature map awal. Dengan cara ini, model dapat menangkap informasi spasial dari berbagai skala tanpa harus meningkatkan ukuran feature map. SPPF membantu model memahami hubungan antara objek dengan lingkungan di sekitarnya, terutama ketika objek memiliki ukuran atau konteks yang berbeda.

```
class SPPF(nn.Module):
    def __init__(self, in_ch, out_ch, k=5):
        super().__init__()
        hidden_ch = in_ch // 2
        self.cv1 = ConvBlock(in_ch, hidden_ch, k=1, s=1, p=0)
        self.cv2 = ConvBlock(hidden_ch * 4, out_ch, k=1, s=1, p=0)
        self.pool = nn.MaxPool2d(
            kernel_size=k,
            stride=1,
            padding=k // 2
        )
    def forward(self, x):
        x = self.cv1(x)
        y1 = self.pool(x)
        y2 = self.pool(y1)
        y3 = self.pool(y2)

        return self.cv2(torch.cat((x, y1, y2, y3), dim=1))
```

6. Backbone

Backbone merupakan bagian utama model yang bertanggung jawab dalam mengekstraksi fitur dari citra input secara bertahap. Proses dimulai dari lapisan convolution awal dengan stride 2 yang berfungsi mengurangi resolusi citra sekaligus meningkatkan jumlah informasi fitur yang dapat diproses pada tahap berikutnya. Selanjutnya, feature map diproses menggunakan beberapa ConvBlock, C3k2, SPPF, dan C2PSA. Semakin dalam proses ekstraksi, resolusi spasial feature map semakin kecil, sedangkan jumlah channel dan tingkat abstraksi fitur semakin besar.

Backbone menghasilkan tiga feature map utama, yaitu P3, P4, dan P5. P3 memiliki resolusi spasial yang lebih tinggi sehingga mampu mempertahankan lebih banyak informasi detail dan digunakan terutama untuk menangkap objek berukuran kecil. P4 memiliki resolusi menengah dan digunakan untuk merepresentasikan objek berukuran sedang. Sementara itu, P5 memiliki resolusi yang lebih rendah tetapi mengandung informasi semantik dan konteks yang lebih luas sehingga sesuai untuk merepresentasikan objek berukuran besar.

```
self.stem = ConvBlock(3, c_stem, k=3, s=2, p=1)

self.l2 = C3k2(c_stem, c_p2, n=n_repeat)

self.l3 = ConvBlock(c_p2, c_p3, k=3, s=2, p=1)

self.l4 = C3k2(c_p3, c_p3, n=n_repeat)

self.l5 = ConvBlock(c_p3, c_p4, k=3, s=2, p=1)

self.l6 = C3k2(c_p4, c_p4, n=n_repeat)

self.l7 = ConvBlock(c_p4, c_p5, k=3, s=2, p=1)

self.l8 = C3k2(c_p5, c_p5, n=n_repeat)

self.l9 = SPPF(c_p5, c_p5, k=5)

self.l10 = C2PSA(c_p5, c_p5)
```

7. Neck FPN dan PAN

Neck digunakan untuk menggabungkan informasi fitur dari berbagai tingkat resolusi. Pada arsitektur ini, neck menggunakan konsep Feature Pyramid Network (FPN) dan Path Aggregation Network (PAN). Proses FPN dimulai dengan melakukan upsampling terhadap feature map P5 menggunakan faktor skala 2. Feature map hasil upsampling kemudian digabungkan dengan P4 sehingga informasi semantik dari layer yang lebih dalam dapat dikombinasikan dengan detail spasial dari layer sebelumnya. Proses tersebut dilanjutkan dengan melakukan upsampling kembali dan menggabungkan hasilnya dengan P3.

Setelah proses upsampling selesai, PAN melakukan proses downsampling menggunakan convolution dengan stride 2. Proses ini mengalirkan kembali informasi dari feature map dengan resolusi tinggi menuju resolusi yang lebih rendah. Dengan demikian, informasi dari P3, P4, dan P5 dapat saling berinteraksi dan menghasilkan feature map yang lebih informatif. Hasil akhir dari proses neck berupa P3_out, P4_out, dan P5_out yang kemudian digunakan oleh detection head dan DepthHead.

8. Detection Head

Detection head bertanggung jawab untuk menghasilkan prediksi objek dari feature map pada tiga skala berbeda. Setiap detection head memiliki dua cabang utama, yaitu cabang regresi dan cabang klasifikasi. Cabang regresi digunakan untuk menghasilkan informasi mengenai posisi dan ukuran bounding box, sedangkan cabang klasifikasi digunakan untuk menentukan kelas dari objek yang terdeteksi.

Penggunaan tiga skala memungkinkan model menangani objek dengan ukuran yang berbeda dalam satu citra. Feature map P3 digunakan terutama untuk mendeteksi objek berukuran kecil karena memiliki resolusi spasial yang lebih tinggi. P4 digunakan untuk objek berukuran sedang, sedangkan P5 digunakan untuk objek berukuran besar dan memiliki informasi semantik yang lebih kuat.

class DetectionHead(nn.Module):

```
def __init__(self, in_ch, num_classes=80):
    super().__init__()
    self.reg = nn.Sequential(
        ConvBlock(in_ch, in_ch, k=3, s=1, p=1),
        nn.Conv2d(in_ch, 4, kernel_size=1),
    )
    self.cls = nn.Sequential(
        ConvBlock(in_ch, in_ch, k=3, s=1, p=1),
        nn.Conv2d(in_ch, num_classes, kernel_size=1),
    )

def forward(self, x):
    return self.reg(x), self.cls(x)
```

9. DepthHead

DepthHead merupakan komponen tambahan pada arsitektur YOLO26n-Depth yang digunakan untuk menghasilkan depth map dari citra RGB. Feature map P3_out, P4_out, dan P5_out terlebih dahulu disesuaikan jumlah channel-nya agar dapat diproses secara bersamaan. Setelah itu, feature map dari ketiga skala tersebut diubah ke ukuran spasial yang sama menggunakan interpolasi bilinear. Ketiga feature map kemudian digabungkan dan diproses melalui beberapa lapisan convolution untuk menghasilkan representasi kedalaman.

Output akhir dari DepthHead memiliki satu channel yang merepresentasikan nilai kedalaman pada setiap piksel. Fungsi aktivasi sigmoid digunakan untuk membatasi nilai output pada rentang 0 sampai 1. Nilai tersebut selanjutnya dapat digunakan sebagai depth map dan divisualisasikan menggunakan gradasi warna untuk menunjukkan perbedaan kedalaman relatif antara area yang lebih dekat dan lebih jauh.

```
class DepthHead(nn.Module):
  def __init__(self, channels=(64, 128, 256)):
    super().__init__()
    self.conv_p3 = ConvBlock(channels[0], 32, k=1, s=1, p=0)
    self.conv_p4 = ConvBlock(channels[1], 32, k=1, s=1, p=0)
    self.conv_p5 = ConvBlock(channels[2], 32, k=1, s=1, p=0)
```

10. Forward Model

Fungsi forward() mengatur alur pemrosesan data mulai dari citra input hingga menghasilkan seluruh output model. Citra input terlebih dahulu diproses oleh backbone untuk menghasilkan feature map P3, P4, dan P5. Ketiga feature map tersebut kemudian diteruskan ke neck FPN dan PAN untuk menggabungkan informasi fitur dari berbagai skala. Proses tersebut menghasilkan P3_out, P4_out, dan P5_out yang memiliki informasi fitur yang lebih lengkap.

Feature map hasil neck kemudian diproses oleh masing-masing detection head untuk menghasilkan prediksi bounding box dan kelas objek. Secara paralel, ketiga feature map tersebut juga diberikan kepada DepthHead untuk menghasilkan depth map. Ukuran depth map kemudian disesuaikan dengan ukuran citra input sehingga posisi setiap nilai kedalaman tetap sesuai dengan posisi piksel pada citra asli.

```
def forward(self, x_list, target_size):
  p3, p4, p5 = x_list


  up_p3 = F.interpolate(
    self.conv_p3(p3),
    size=target_size,
    mode='bilinear',
    align_corners=False
```

```
    )
    up_p4 = F.interpolate(
        self.conv_p4(p4),
        size=target_size,
        mode='bilinear',
        align_corners=False
    )
    up_p5 = F.interpolate(
        self.conv_p5(p5),
        size=target_size,
        mode='bilinear',
        align_corners=False
    )

    fused = torch.cat([up_p3, up_p4, up_p5], dim=1)
    return self.refine(fused)
```

11. Output Model

Model menghasilkan output dalam bentuk dictionary yang berisi hasil deteksi objek dan estimasi kedalaman. p3_bbox, p4_bbox, dan p5_bbox berisi hasil prediksi bounding box pada tiga skala feature map. Sementara itu, p3_cls, p4_cls, dan p5_cls berisi hasil prediksi kelas objek pada masing-masing skala. Key depth berisi depth map yang dihasilkan oleh DepthHead.

Struktur output tersebut memungkinkan hasil deteksi dan estimasi kedalaman diakses secara terpisah. Data deteksi dapat digunakan untuk mengetahui keberadaan, lokasi, dan kelas objek, sedangkan data depth digunakan untuk memperoleh informasi kedalaman pada setiap piksel citra. Dengan demikian, model tidak hanya melakukan deteksi objek, tetapi juga melakukan estimasi kedalaman secara bersamaan dalam satu proses inferensi.

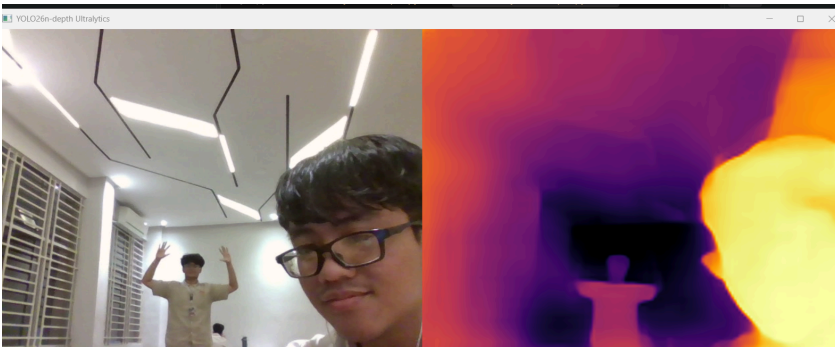
```
return {
    'p3_bbox': p3_reg,
    'p3_cls': p3_cls,
    'p4_bbox': p4_reg,
    'p4_cls': p4_cls,
    'p5_bbox': p5_reg,
    'p5_cls': p5_cls,
    'depth': depth_map,
}
```

KELOMPOK 1.1
Lightweight YOLO26-Depth

NAMA: Lutfi Maulana (NIM. 245150301111021)
TUGAS: Mengidentifikasi dan membedah kelemahan teknis model baseline serta merumuskan rekomendasi arah perbaikan (Sub-bab 4.2 dan 4.3).

NAMA : Muhammad Dzakwan Ikram (NIM. 245150301111019)
TUGAS: Menyusun evaluasi kinerja kuantitatif model, analisis visual peta kedalaman, dan tabel ringkasan metrik (Sub-bab 4.1).

4.1 Evaluasi Kinerja Kuantitatif dan Analisis Visualisasi Output



Sumber: Penulis

Gambar 4.1 Hasil Output Depth Map

Secara visual pada Gambar 4.1, model berhasil memetakan jarak secara bertingkat menggunakan skema warna inferno:

- **Objek depan (jarak dekat):** Wajah dan kepala pengguna di bagian depan tampak berwarna kuning hingga oranye cerah, menandakan jarak paling dekat dengan kamera.
- **Objek menengah:** Orang di sebelah kiri yang posisinya lebih mundur terbaca dengan warna merah keunguan, menunjukkan model mengenali perbedaan bidang jarak antar-orang.
- **Latar belakang (jarak jauh):** Area tembok, jendela, dan plafon diberi warna ungu tua hingga hitam, menandakan batas terjauh dari pandangan kamera.

Pengujian kuantitatif terhadap model baseline YOLO26n-Depth dilakukan menggunakan data validasi dari dataset NYU Depth V2 setelah melalui proses pelatihan selama 30 epoch. Untuk mengukur keandalan dan presisi prediksi jarak yang dihasilkan oleh model, digunakan empat parameter evaluasi standar:

- **Threshold Accuracy ($\delta 1$ ($\delta < 1.25$)):** Mengukur persentase piksel yang memiliki rasio deviasi antara jarak prediksi dan jarak sebenarnya di bawah batas toleransi 25%. Semakin tinggi nilai $\delta 1$ (mendekati 1 atau 100%),

semakin banyak titik kedalaman yang terprediksi secara akurat.

- **Absolute Relative Error (Abs Rel):** Mengukur rata-rata persentase kesalahan relatif antara kedalaman hasil estimasi model dengan kedalaman sebenarnya di lapangan. Semakin kecil nilai galat ini, semakin rendah tingkat kesalahan prediksi model.
- **Root Mean Squared Error (RMSE):** Menghitung akar kuadrat dari rata-rata selisih kuadrat jarak untuk melihat simpangan kesalahan fisik model secara keseluruhan dalam satuan meter. Metrik ini memberi bobot penalti lebih besar pada nilai prediksi yang melenceng jauh.
- **Scale-Invariant Logarithmic Error (SILog):** Mengukur variansi kesalahan prediksi pada skala logaritmik secara independen terhadap pergeseran skala jarak global kamera. Nilai SILog yang rendah menunjukkan bahwa hierarki ruang dan kedalaman relatif antarobjek di dalam satu ruangan tetap konsisten.

Capaian nilai evaluasi kuantitatif dari model baseline YOLO26n-Depth berdasarkan keempat metrik tersebut dirangkum pada Tabel 4.1.

Tabel 4.1 Hasil Evaluasi Kuantitatif Model Baseline YOLO26n-Depth (30 Epoch)

Metrik Evaluasi	Nilai Uji	Keterangan dan Makna Teknis
$\delta 1$ ($\delta < 1.25$)	0,8755 (87,55%)	Persentase piksel dengan estimasi akurat; 87,55% titik kedalaman memiliki simpangan rasio di bawah 25% dari nilai sebenarnya
Abs Rel	0,1166 (11,66%)	Rata-rata persentase kesalahan relatif jarak terhadap kondisi riil tergolong rendah
RMSE	0,4538 m	Simpangan kesalahan fisik jarak prediksi rata-rata menyimpang sekitar 0,45 meter dari posisi

		objek nyata.
SILog	13,7053	Ukuran variansi kesalahan logaritmik; menunjukkan kestabilan prediksi jarak tanpa terganggu pergeseran skala global kamera

Sumber: Penulis

4.2 Analisis Kelemahan dan Keterbatasan Model Baseline

Berdasarkan hasil visualisasi inferensi pada Gambar 4.1 dan implementasi kode pada bab-bab sebelumnya, model baseline YOLO26n-Depth masih memiliki sejumlah kelemahan dan keterbatasan teknis:

- **Garis Pinggir Objek Masih Buram (*Edge Bleeding*):** Batas antara tubuh subjek dan dinding belakang tampak kabur serta melebar karena model varian nano kehilangan detail kecil saat mengecilkan ukuran gambar.
- **Kotak Deteksi (*Bounding Box*) Belum Muncul:** Walaupun arsitekturnya dirancang untuk mendeteksi objek sekaligus mengukur kedalaman, program saat ini baru menampilkan peta kedalaman saja di layar.
- **Benda Ramping Tidak Terbaca:** Objek tipis seperti jeruji jendela di sisi kiri dan garis lampu plafon tidak terdeteksi, melainkan hanya dianggap sebagai permukaan tembok yang rata.
- **Warna Tampilan Mudah Berubah (*Flickering*):** Nilai warna dihitung ulang di setiap frame secara mandiri, sehingga gerakan orang atau perubahan cahaya sedikit saja membuat gradasi warna satu ruangan langsung bergeser drastis.
- **Ukuran Jarak Masih Relatif:** Hasil keluaran model masih berada pada rentang skala 0 sampai 1 dan belum menunjukkan ukuran jarak riil dalam satuan meter.

4.3 Rekomendasi Solusi dan Arah Perbaikan

Untuk memperbaiki berbagai kekurangan pada model baseline tersebut, beberapa langkah pengembangan yang dapat dilakukan meliputi:

- **Memasang Modul Perhatian (CBAM):** Menambahkan modul CBAM (seperti yang dibahas pada Bab 1) agar AI lebih fokus melihat garis pinggir benda dan objek tipis, sehingga batas gambar tidak tampak buram lagi.
- **Memunculkan Kotak Deteksi Objek:** Memperbarui tampilan kamera agar tidak hanya memperlihatkan warna peta kedalaman, tetapi juga langsung menampilkan kotak penanda objek dan nama kelasnya di layar.
- **Membuat Warna Tampilan Lebih Stabil:** Menghaluskan perubahan warna antar-frame agar tampilan video tidak mudah berkedip saat orang di depan kamera bergerak atau saat pencahayaan ruangan berubah.
- **Mengubah Nilai Jarak ke Satuan Meter:** Melatih model lebih lanjut agar angka kedalaman yang dihasilkan bisa langsung terbaca dalam ukuran meter asli, bukan sekadar nilai perkiraan antara 0 sampai 1.

DAFTAR PUSTAKA

Ardhi, O.D.W., Hartono, R. dan Yoeseph, N.M. (2026) 'Plant Disease Object Detection on PlantDoc Using YOLO26n', *SISFO Jurnal Ilmiah*, 10(1), hlm. 187–197. doi: 10.29103/sisfo.v10i1.27063.

Burukanli, M., Cibuk, M. dan Ari, D. (2026) 'Attention-Enhanced YOLOv26 with Tree-Structured Parzen Estimator Optimization for Robust Dental Surgical Tool Detection', *Applied Sciences*, 16(15), hlm. 7654. doi: 10.3390/app16157654.

Liu, S., Chen, R., Ye, M., Luo, J., Yang, D. dan Dai, M. 2024. EcoDetect-YOLO: A Lightweight, High-Generalization Methodology for Real-Time Detection of Domestic Waste Exposure in Intricate Environmental Landscapes. Sensors, 24(14), hal. 4666. https://doi.org/10.3390/s24144666

Ning, J., Liu, B., Zhou, B., Zhang, J., Ding, Y., Li, Y., Gao, Y. dan Ma, Y. 2026. An Attention-Guided Lightweight YOLO11n with Ghost Convolution for Pediatric Tooth Detection in Panoramic Radiographs. https://doi.org/10.18280/ts.430218

Redmon, J., Divvala, S., Girshick, R., dan Farhadi, A., 2016. You only look once: Unified, real-time object detection. Dalam Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 779–788

Ultralytics. 2026. *YOLO26*. Ultralytics Documentations. Tersedia di: https://docs.ultralytics.com/models/yolo26

Yuan, Z., Tang, X., Ning, H. dan Yang, Z. 2024. LW-YOLO: Lightweight Deep Learning Model for Fast and Precise Defect Detection in Printed Circuit Boards. *Symmetry*, 16(4), hal. 418. https://doi.org/10.3390/sym16040418

Zare, O., Beigzadeh, M., Bzuayene, A.A., Arslan, E., Haghvirdizadeh, H., Rad, S.Y.B., Abbasi, A., Joloudari, J.H. dan Gaftandzhieva, S. 2026. LightAttn-YOLO-V8: an efficient colorectal polyp detection with lightweight attention mechanism based on YOLO. Scientific Reports. https://doi.org/10.1038/s41598-026-35390-z

Hal yang harus ditambahkan:

1. Benchmark Kecepatan & Latensi (Real-time Performance)

- Dokumen saat ini baru menampilkan evaluasi akurasi kedalaman ($\delta 1$, Abs Rel, RMSE, SILog).
- Karena modul berfokus pada arsitektur *lightweight*, tambahkan tabel pengujian performa komputasi *real-time*, seperti **FPS (Frames Per Second)**, **Inference Latency (ms)**, serta konsumsi **RAM/GPU Memory** saat skrip dijalankan di PC/Webcam.

2. Formula & Penjelasan Loss Function

- Bab 3 sudah membedah struktur modul PyTorch (DetectionHead dan DepthHead), namun belum menjelaskan fungsi kerugian (*Loss Function*) yang digunakan saat proses *training* 30 epoch.
- Tambahkan penjelasan *Multitask Loss* yang menggabungkan *Detection Loss* (BCE/CIoU) dengan *Depth Estimation Loss* (seperti *Scale-Invariant Logarithmic Loss* atau *BerHu Loss*).

3. Panduan Ekspor & Kuantisasi Model Mobile

- Bab 1 menyebutkan kompatibilitas ekosistem mobile (LiteRT, NCNN, CoreML) dan kuantisasi FP16/INT8, tetapi Bab 2 baru menyediakan skrip PyTorch/OpenCV untuk desktop.
- Tambahkan sub-bab atau lampiran kode untuk proses ekspor bobot .pt ke format .onnx atau .tflite, beserta contoh fungsi kuantisasi model agar alur *deployment* ke HP menjadi utuh.

4. Rumus Konversi Kedalaman Relatif ke Jarak Metrik

- Pada kelemahan model, disebutkan bahwa nilai luaran masih berupa rentang relatif $[0, 1]$.

- Masukkan persamaan matematika atau metode kalibrasi kamera (menggunakan *focal length* f, Z_{min}, dan Z_{max}) untuk menjelaskan cara mengubah nilai piksel $[0, 1]$ menjadi estimasi jarak riil dalam satuan meter/centimeter.

5. Kode Visualisasi Gabungan (Bounding Box + Overlay Depth)

- Dokumen mencatat bahwa *bounding box* objek belum dimunculkan pada program visualisasi.
- Tambahkan potongan kode *post-processing* yang menggabungkan keluaran deteksi objek (p3_bbox, p4_bbox, p5_bbox) dengan bacaan titik kedalaman, sehingga tampilan kamera bisa langsung memuat kotak penanda objek beserta angka jaraknya.

Deskripsi Tugas: Porting YOLO26n-Depth ke Mobile & Output Jarak Metrik (Meter/cm)

Target Output & Deliverables Utama

- **1. Berkas Model Format Mobile (.tflite / .onnx / .ncnn)**
  - Model PyTorch (.pt) berhasil diekspor ke format ONNX dan disesuaikan menjadi format **LiteRT (TFLite)** atau **NCNN**.
  - Model telah dioptimasi/dikuantisasi (FP16 atau INT8) agar *lightweight* dan tidak memicu *overheating* di HP.
- **2. Logika Pengukuran Jarak Metrik (Meters & Centimeters)**
  - Fungsi konversi nilai *depth map* relatif $[0, 1]$ menjadi angka riil satuan meter (m) dan centimeter (cm).

    - Fungsi ekstraksi nilai kedalaman pada titik tengah (*center point*) *bounding box* objek yang terdeteksi.
    - *(Opsional)* Fitur *Tap to Measure*: Pengguna bisa menyentuh titik mana saja di layar HP untuk melihat angka jarak piksel tersebut.
- **3. Integrasi Pipeline Kamera Mobile**
    - Aplikasi membaca *frame* kamera HP secara *real-time* dan melakukan inferensi model tanpa *lag* mendasar (minimal 15–30 FPS).
    - Pengolahan *preprocessing* (resize/normalize) dan *postprocessing* berjalan langsung di dalam perangkat (*on-device inference*).
- **4. Tampilan Layar (UI Visual Overlay)**
    - **Objek & Jarak**: *Bounding box* objek muncul di layar beserta label nama kelas dan angka jarak metrik (contoh: **"Orang - 1.45 m / 145 cm"**).
    - **Visual Peta Kedalaman**: Opsi tampilan *side-by-side* atau *toggle* peta warna (*Inferno colormap*) untuk melihat kontur jarak visual secara langsung.

Kriteria Selesai (Definition of Done)

- [ ] File model siap pakai (.tflite / .onnx / .param & .bin).
- [ ] *Source code* repositori aplikasi mobile (Flutter / React Native / Android Native / iOS Native).
- [ ] File installer (APK / TestFlight) atau **video demo singkat** yang menampilkan kamera HP mendeteksi objek beserta angka jarak riilnya (meter/cm) di layar.

File PPT (canva):

https://canva.link/122zc0a86io126a

Saran isi Slide:

Slide 1: Judul & Anggota Kelompok

- **Judul**: Computer Vision - Analisis Mendalam Topik Lightweight YOLO26n-Depth
- **Dosen Pengampu**: Prof. Dr. Eng. Fitri Utaminingrum, S.T., M.T.
- **Anggota Kelompok**:
    - Lutfi Maulana (NIM. 245150301111021)
    - Muhammad Dzakwan Ikram (NIM. 245150301111019)

Slide 2: Konsep Utama & Keunggulan YOLO26n-Depth

- **Monocular Depth Estimation**: Mengukur jarak objek langsung dari satu kamera RGB tanpa perlu LiDAR atau sensor tambahan.
- **Skala Unbounded Depth**: Mampu memprediksi jarak dekat (*indoor*) maupun jarak jauh (*outdoor*) menggunakan skala logaritmik.
- **Desain Lightweight**: Ukuran model di bawah 10 juta parameter, hemat daya, serta mendukung ekspor ke format mobile (LiteRT, NCNN, CoreML).

Slide 3: Dataset Pelatihan Model

- **NYU Depth V2**: Dataset citra *indoor* untuk melatih estimasi jarak objek ruangan (meja, kursi, dinding).
- **KITTI Vision Benchmark**: Dataset citra *outdoor* jalan raya untuk melatih pengukuran jarak skala luas.
- **MS COCO**: Digunakan pada tahap *pre-training backbone* untuk klasifikasi dan deteksi objek.

Slide 4: Spesifikasi Arsitektur Baseline (YOLO26n)

- **Konfigurasi Compound Scaling**: *Depth factor* 0.50, *width factor* 0.25 (260 layer, 2.57M parameter, 6.1 GFLOPs).

- **Struktur Utama**:
    - **Backbone**: Conv, C3k2, SPPF, C2PSA untuk ekstraksi fitur multi-skala (P3, P4, P5).
    - **Neck**: FPN + PAN untuk menggabungkan informasi detail spasial dan semantik.
    - **Detection Head**: Menghasilkan *bounding box* dan klasifikasi secara langsung (*NMS-Free*).

Slide 5: Korelasi Arsitektur & Kode Program (Backbone & Neck)

- **ConvBlock**: Lapisan dasar pengekstrak fitur (Conv2d + BatchNorm2d + SiLU).
- **C3k2 & Bottleneck**: Blok utama Cross Stage Partial untuk ekstraksi fitur efisien dengan koneksi residual.
- **C2PSA**: Lapisan *attention* internal pada ujung *backbone* untuk menangkap hubungan spasial.
- **SPPF**: Memperluas area pandang (*receptive field*) menggunakan *multi-scale pooling*.

Slide 6: Korelasi Arsitektur & Kode Program (Head & Output)

- **DetectionHead**: Memiliki dua cabang paralel untuk regresi posisi *bounding box* dan skor kelas.
- **DepthHead**: Menggabungkan peta fitur P3, P4, P5 via interpolasi bilinear untuk memprediksi *depth map* 1-channel.
- **Fungsi forward()**: Mengembalikan *dictionary* berisi hasil deteksi (p3_bbox, p4_bbox, dll.) dan *peta kedalaman* (depth).

Slide 7: Arsitektur Tambahan & Mekanisme Peta Kedalaman

- **Dynamic Attention (CBAM)**: Menjaga akurasi tepi objek melalui *Channel Attention* dan *Spatial Attention*.
- **Normalisasi (normalize_depth)**: Pemotongan persentil 2%–98% untuk meredam pantulan silau/bayangan.

- **Visualisasi Warna (colorize_depth)**: Menggunakan COLORMAP_INFERNO (Kuning/Oranye = Dekat, Ungu/Hitam = Jauh).

Slide 8: Hasil Eksperimen & Evaluasi Kuantitatif

- **Hasil Visual**: Model berhasil membedakan hirarki jarak antara subjek depan, orang di belakang, dan dinding.
- **Metrik Evaluasi (NYU Depth V2, 30 Epoch)**:
  - $\delta 1 (\delta < 1.25)$: 87.55% (Akurasi rasio deviasi tinggi)
  - **Abs Rel**: $0.1166 / 11.66\%$ (Galat kesalahan relatif rendah)
  - **RMSE**: $0.4538m$ (Simpangan jarak fisik sekitar $0.45meter$)
  - **SILog**: 13.7053 (Kestabilan hirarki kedalaman logaritmik)

Slide 9: Kelemahan & Keterbatasan Model Baseline

- **Garis Pinggir Buram (*Edge Bleeding*)**: Terjadi penuangan warna pada batas tepi subjek.
- **Kotak Deteksi Belum Tampil**: Tampilan program baru memuat *depth map* tanpa *bounding box* objek.
- **Objek Ramping Tidak Terbaca**: Garis lampu dan jeruji jendela menyatu dengan latar belakang.
- **Skala Relatif & Berkedip (*Flickering*)**: Nilai masih rentang $[0, 1]$ dan gradasi warna mudah bergeser.

Slide 10: Solusi Perbaikan & Pengembangan Lanjutan

- **Integrasi CBAM**: Memfokuskan model pada kontur dan tepi objek tipis.
- **Visualisasi Gabungan**: Memunculkan *bounding box*, nama kelas, dan label jarak pada layar.
- **Stabilisasi Warna**: Menghaluskan perubahan gradasi warna antar-*frame*.

- **Konversi Jarak Metrik**: Melatih ulang (*retrain*) model agar menghasilkan output jarak riil dalam satuan meter/cm.